Технологии конструирования программного обеспечения

Отчет по лабораторной работе № 2

Группа: 241-327 Студент: Дубина Анастасия Александровна

Задание на лабораторную работу

11. Двигатель, бензиновый двигатель, дизельный двигатель, реактивный двигатель.

Диаграмма классов

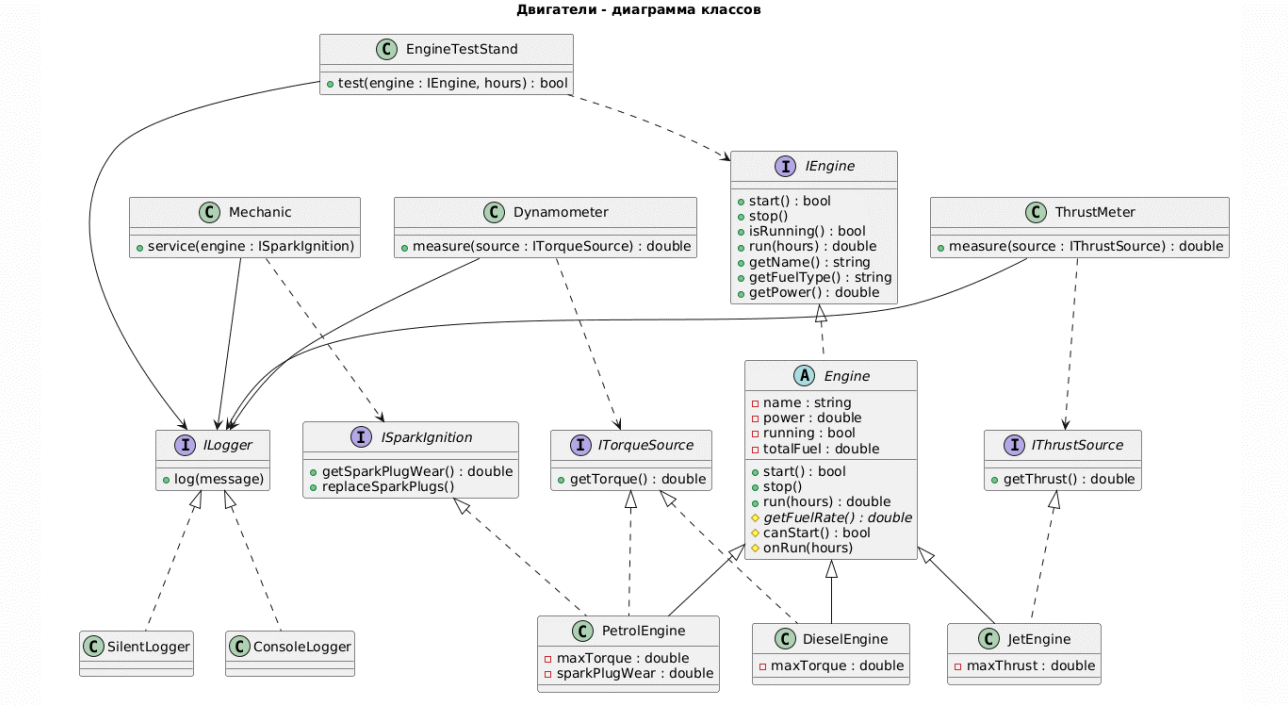
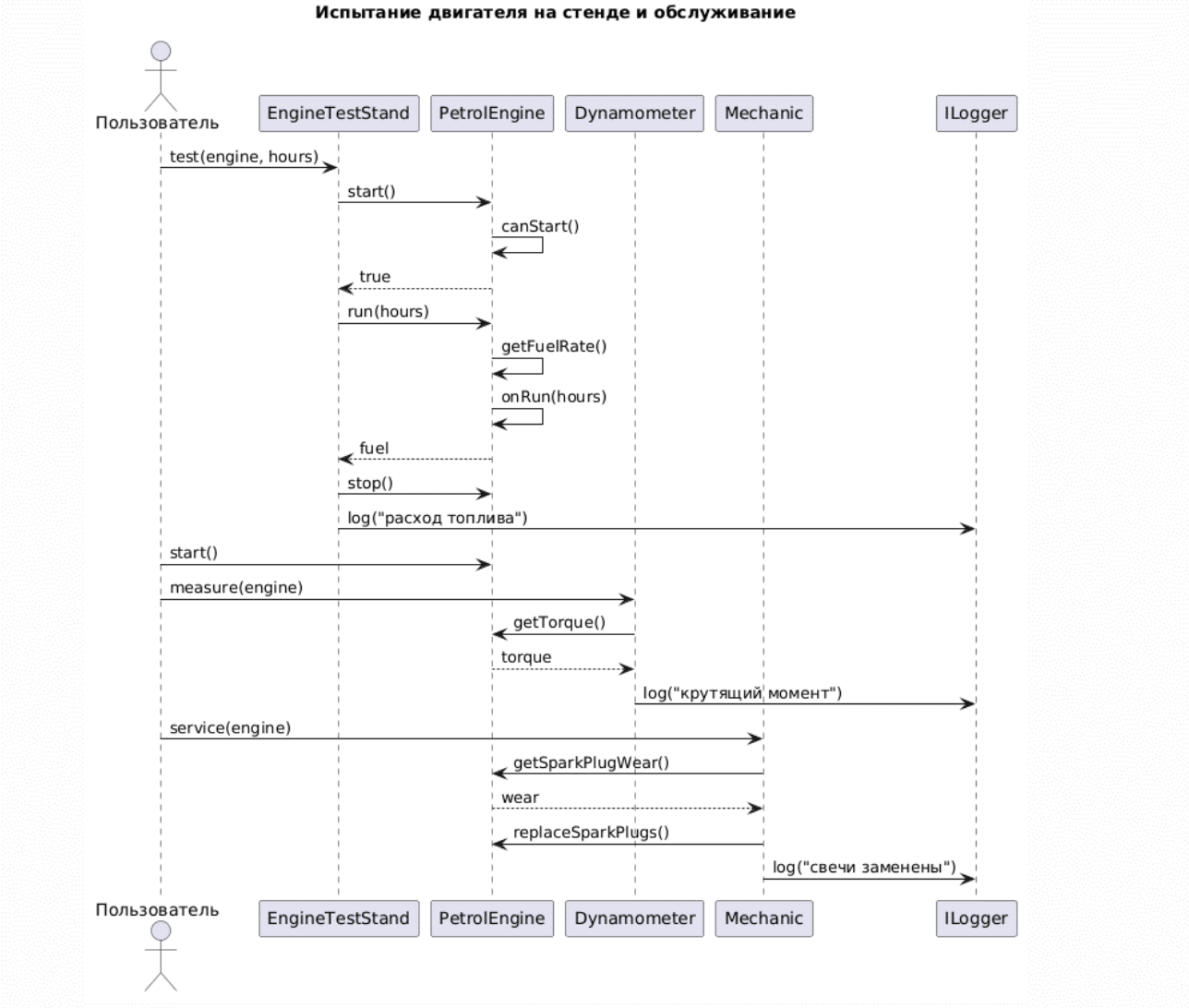


Диаграмма последовательности



Исходный код программы

```
#include <iostream>
#include <string>
#include <vector>
#include <clocale>
#include <sstream>
#include <iomanip>
#include <cmath>

using namespace std;

// Форматирование числа: 12.5 -> "12.50"
string num(double value) {
   ostringstream out;
   out << fixed << setprecision(2) << value;
```

```
    return out.str();
}

// ====================== Интерфейсы (ISP) ======================

// Общий интерфейс любого двигателя: только то, что есть у всех двигателей.
class IEngine {
public:
    virtual ~IEngine() {}
    virtual bool start() = 0;
    virtual void stop() = 0;
    virtual bool isRunning() const = 0;
    virtual double run(double hours) = 0;   // возвращает израсходованное топливо, л
    virtual string getName() const = 0;
    virtual string getFuelType() const = 0;
    virtual double getPower() const = 0;    // мощность, кВт
};

// Крутящий момент есть только у поршневых двигателей.
class ITorqueSource {
public:
    virtual ~ITorqueSource() {}
    virtual double getTorque() const = 0;   // Н*м
};

// Реактивная тяга есть только у реактивного двигателя.
class IThrustSource {
public:
    virtual ~IThrustSource() {}
    virtual double getThrust() const = 0;   // кН
};

// Свечи зажигания есть только у бензинового двигателя.
class ISparkIgnition {
public:
    virtual ~ISparkIgnition() {}
    virtual double getSparkPlugWear() const = 0;   // износ, %
    virtual void replaceSparkPlugs() = 0;
};

// Вывод сообщений: высокоуровневые классы зависят от абстракции (DIP).
class ILogger {
public:
    virtual ~ILogger() {}
    virtual void log(const string& message) = 0;
};
```

```
class ConsoleLogger : public ILogger {
public:
   void log(const string& message) override {
      cout << "[LOG] " << message << endl;
   }
};

// ======================== Базовый класс (LSP) ======================

// Общая логика всех двигателей вынесена в базовый класс.
// Наследники меняют только детали (расход топлива, условие запуска),
// но не нарушают контракт: start() возвращает false, если запуск невозможен,
// run() у заглушенного двигателя ничего не расходует.
class Engine : public IEngine {
public:
   Engine(const string& name, double power)
      : name(name), power(power), running(false), totalFuel(0) {}

   bool start() override {
      if (running || !canStart()) return false;
      running = true;
      return true;
   }

   void stop() override { running = false; }

   bool isRunning() const override { return running; }

   double run(double hours) override {
      if (!running || hours <= 0) return 0;
      double used = getFuelRate() * hours;
      totalFuel += used;
      onRun(hours);
      return used;
   }

   string getName() const override { return name; }
   double getPower() const override { return power; }
   double getTotalFuel() const { return totalFuel; }

protected:
   virtual double getFuelRate() const = 0;      // расход, л/ч
   virtual bool canStart() const { return true; }
   virtual void onRun(double) {}

private:
   string name;
```

```
    double power;
    bool running;
    double totalFuel;
};

// ====================== Конкретные двигатели ======================

class PetrolEngine : public Engine, public ITorqueSource, public ISparkIgnition {
public:
    PetrolEngine(const string& name, double power, double maxTorque)
        : Engine(name, power), maxTorque(maxTorque), sparkPlugWear(0) {}

    string getFuelType() const override { return "бензин"; }
    double getTorque() const override { return isRunning() ? maxTorque : 0; }

    double getSparkPlugWear() const override { return sparkPlugWear; }
    void replaceSparkPlugs() override { sparkPlugWear = 0; }

protected:
    double getFuelRate() const override { return getPower() * 0.12; }
    // С изношенными свечами двигатель не заводится
    bool canStart() const override { return sparkPlugWear < 100; }
    void onRun(double hours) override {
        sparkPlugWear += hours * 2;
        if (sparkPlugWear > 100) sparkPlugWear = 100;
    }

private:
    double maxTorque;
    double sparkPlugWear;
};

class DieselEngine : public Engine, public ITorqueSource {
public:
    DieselEngine(const string& name, double power, double maxTorque)
        : Engine(name, power), maxTorque(maxTorque) {}

    string getFuelType() const override { return "дизель"; }
    double getTorque() const override { return isRunning() ? maxTorque : 0; }

protected:
    double getFuelRate() const override { return getPower() * 0.09; }

private:
    double maxTorque;
};
```

```
class JetEngine : public Engine, public IThrustSource {
public:
   JetEngine(const string& name, double power, double maxThrust)
      : Engine(name, power), maxThrust(maxThrust) {}

   string getFuelType() const override { return "керосин"; }
   double getThrust() const override { return isRunning() ? maxThrust : 0; }

protected:
   double getFuelRate() const override { return getPower() * 0.3; }

private:
   double maxThrust;
};

// ======================== Высокоуровневые классы (DIP) ========================

// Испытательный стенд работает с любым двигателем через IEngine (LSP)
// и не знает о конкретных классах.
class EngineTestStand {
public:
   EngineTestStand(ILogger& logger) : logger(logger) {}

   bool test(IEngine& engine, double hours) {
      if (!engine.start()) {
         logger.log(engine.getName() + ": не удалось запустить");
         return false;
      }
      double fuel = engine.run(hours);
      engine.stop();
      logger.log(engine.getName() + " (" + engine.getFuelType() + ", " +
            num(engine.getPower()) + " кВт): " + num(hours) + " ч, расход " +
            num(fuel) + " л");
      return true;
   }

private:
   ILogger& logger;
};

// Динамометр измеряет крутящий момент: ему нужен только ITorqueSource (ISP).
class Dynamometer {
public:
   Dynamometer(ILogger& logger) : logger(logger) {}

   double measure(const ITorqueSource& source) {
      double torque = source.getTorque();
```

```
        logger.log("Динамометр: крутящий момент " + num(torque) + " Н*м");
        return torque;
    }

private:
    ILogger& logger;
};

// Стенд тяги: ему нужен только IThrustSource (ISP).
class ThrustMeter {
public:
    ThrustMeter(ILogger& logger) : logger(logger) {}

    double measure(const IThrustSource& source) {
        double thrust = source.getThrust();
        logger.log("Стенд тяги: тяга " + num(thrust) + " кН");
        return thrust;
    }

private:
    ILogger& logger;
};

// Механик меняет свечи: ему нужен только ISparkIgnition (ISP).
class Mechanic {
public:
    Mechanic(ILogger& logger) : logger(logger) {}

    void service(ISparkIgnition& engine) {
        if (engine.getSparkPlugWear() >= 50) {
            engine.replaceSparkPlugs();
            logger.log("Механик: свечи заменены");
        } else {
            logger.log("Механик: замена свечей не требуется (износ " +
                    num(engine.getSparkPlugWear()) + "%)");
        }
    }

private:
    ILogger& logger;
};

// ====================== Тестирование ======================

class SilentLogger : public ILogger {
public:
    void log(const string&) override {}
```

```
};

class Tester {
public:
   Tester() : passed(0), failed(0) {}

   void check(bool condition, const string& name) {
      if (condition) {
         passed++;
         cout << "  [OK]   " << name << endl;
      } else {
         failed++;
         cout << "  [FAIL] " << name << endl;
      }
   }

   void printResult() const {
      cout << "Пройдено: " << passed << ", провалено: " << failed << endl;
   }

private:
   int passed;
   int failed;
};

void testStartStop(Tester& t) {
   DieselEngine engine("Д-1", 100, 400);
   t.check(!engine.isRunning(), "Новый двигатель заглушен");
   t.check(engine.start() && engine.isRunning(), "Запуск двигателя");
   t.check(!engine.start(), "Повторный запуск запрещен");
   engine.stop();
   t.check(!engine.isRunning(), "Остановка двигателя");
}

// Сравнение дробных чисел с допуском
bool equal(double a, double b) { return fabs(a - b) < 1e-9; }

void testFuel(Tester& t) {
   DieselEngine diesel("Д-1", 100, 400);
   t.check(diesel.run(1) == 0, "Заглушенный двигатель не расходует топливо");
   diesel.start();
   t.check(equal(diesel.run(2), 18), "Расход дизеля: 100 кВт * 0.09 * 2 ч = 18 л");

   JetEngine jet("Р-1", 1000, 50);
   jet.start();
   t.check(equal(jet.run(1), 300), "Расход реактивного: 1000 кВт * 0.3 * 1 ч = 300 л");
}
```

```
// LSP: любой наследник Engine подставляется вместо IEngine
// и ведет себя по одному контракту.
void testSubstitution(Tester& t) {
   PetrolEngine petrol("Б-1", 100, 200);
   DieselEngine diesel("Д-1", 100, 400);
   JetEngine jet("Р-1", 1000, 50);
   vector<IEngine*> engines = { &petrol, &diesel, &jet };

   bool allOk = true;
   for (IEngine* e : engines) {
      bool started = e->start();
      double fuel = e->run(1);
      e->stop();
      if (!started || fuel <= 0 || e->isRunning()) allOk = false;
   }
   t.check(allOk, "Все двигатели одинаково работают через IEngine");

   SilentLogger logger;
   EngineTestStand stand(logger);
   t.check(stand.test(petrol, 1) && stand.test(diesel, 1) && stand.test(jet, 1),
         "Стенд испытывает любой двигатель");
}

void testTorqueAndThrust(Tester& t) {
   SilentLogger logger;
   Dynamometer dyno(logger);
   ThrustMeter thrustMeter(logger);
   PetrolEngine petrol("Б-1", 100, 200);
   DieselEngine diesel("Д-1", 100, 400);
   JetEngine jet("Р-1", 1000, 50);

   t.check(dyno.measure(diesel) == 0, "У заглушенного двигателя нет момента");
   petrol.start();
   diesel.start();
   jet.start();
   t.check(dyno.measure(petrol) == 200, "Момент бензинового двигателя");
   t.check(dyno.measure(diesel) == 400, "Момент дизельного двигателя");
   t.check(thrustMeter.measure(jet) == 50, "Тяга реактивного двигателя");
}

void testSparkPlugs(Tester& t) {
   SilentLogger logger;
   Mechanic mechanic(logger);
   PetrolEngine petrol("Б-1", 100, 200);

   petrol.start();
```

```
    petrol.run(30);
    t.check(petrol.getSparkPlugWear() == 60, "Свечи изнашиваются при работе");
    petrol.run(30);
    petrol.stop();
    t.check(!petrol.start(), "С изношенными свечами двигатель не заводится");
    mechanic.service(petrol);
    t.check(petrol.getSparkPlugWear() == 0 && petrol.start(), "После замены свечей двигатель заводится");
}

void runTests() {
    Tester t;
    cout << "\n===== Тестирование =====" << endl;
    testStartStop(t);
    testFuel(t);
    testSubstitution(t);
    testTorqueAndThrust(t);
    testSparkPlugs(t);
    t.printResult();
}

// ======================== Демонстрация ========================

int main() {
    setlocale(LC_ALL, "Russian");

    ConsoleLogger logger;
    EngineTestStand stand(logger);
    Dynamometer dyno(logger);
    ThrustMeter thrustMeter(logger);
    Mechanic mechanic(logger);

    PetrolEngine petrol("ВАЗ-21126", 72, 140);
    DieselEngine diesel("КАМАЗ-740", 191, 833);
    JetEngine jet("АЛ-31Ф", 20000, 123);

    cout << "===== Демонстрация =====" << endl;

    // LSP: стенд получает разные двигатели через общий интерфейс
    vector<IEngine*> engines = { &petrol, &diesel, &jet };
    for (IEngine* e : engines) {
        stand.test(*e, 2);
    }

    // ISP: каждому измерителю передается только нужный ему интерфейс
    petrol.start();
    diesel.start();
    jet.start();
```

```
    dyno.measure(petrol);
    dyno.measure(diesel);
    thrustMeter.measure(jet);

    petrol.run(25);
    petrol.stop();
    diesel.stop();
    jet.stop();
    mechanic.service(petrol);

    runTests();
    return 0;
}
```